Multimodal video analysis application

Multimodal video analysis application

- 1. Concept introduction
  - 1.1 What is "video analysis"?
  - 1.2 Brief description of the implementation principle
- 2. Code analysis
  - Key code
    - 1. The tool layer entry (`largemodel/utils/tools_manager.py`)
    - 2. The model interface layer and the frame extraction (`largemodel/utils/large_model_interface.py`)
  - Code analysis
- 3. Hands-on operation
  - 3.1 Configure the large model
    - 3.1.1 Configure the LLM platform (`yahboom.yaml`)
    - 3.1.2 Configure the model interface (`large_model_interface.yaml`)
  - 3.2 Start it and test the function (text input mode)
  - 3.3 Voice interaction
- 4. Frequently asked questions and solutions
  - Problem 1: it reports "the video file cannot be found" or "the keyframes cannot be extracted from the video".
  - Problem 2: analysing a longer video takes a very long time.

> **Hardware requirement: a Jetson with a 4GB configuration cannot run the cases of this tutorial; please use a Jetson with 8GB of memory or more.**

---

1. Concept introduction

1.1 What is "video analysis"?

In the `largemodel` project, the **multimodal video analysis** function lets the robot process a video and summarise its core content in natural language, describe the key events or answer specific questions about the video. This lifts the robot from only understanding static pictures to understanding a dynamic world that sometimes has a temporal structure.

The core tool of this function is `analyze_video`. When the user provides a video file and asks a question (for example "summarise what this video is about"), the system calls this tool, processes and analyses the video, and returns the textual answer of the AI.

1.2 Brief description of the implementation principle

The core challenge of the offline video analysis is how to let the model process efficiently a video that contains hundreds or thousands of frames. One mainstream implementation principle is the following:

1. **Keyframe extraction**: first of all, the system does not process every frame of the video, but extracts the most representative **keyframes** with an algorithm (such as the scene boundary detection or the sampling at a fixed time interval). This greatly reduces the amount of data that has to be processed.
2. **Image encoding**: each extracted keyframe is sent into a vision encoder, just like in the visual understanding application, and is converted into a numeric vector that contains the image information.

3. **Fusion of the temporal information**: this is the biggest difference from the understanding of a single picture. The model has to understand the temporal order of these keyframes. A recurrent neural network (RNN) or a Transformer model is normally used to fuse the vectors of all the keyframes and to form a "memory" vector that represents the dynamic content of the whole video.
4. **Question answering and generation**: the text question of the user is encoded and then fused across the modalities with this "memory" vector. Finally, the language model part generates the summary of the whole video or the answer to the specific question based on this fused information.

Put simply, it **"condenses" a piece of video into a few key pictures and their order, and then understands the whole story like a comic strip and answers the related questions**.

---

2. Code analysis

Key code

1. The tool layer entry (`largemodel/utils/tools_manager.py`)

The `analyze_video` function in this file defines the execution flow of the tool.

```
# From largemodel/utils/tools_manager.py
class ToolsManager:
    # ...
    def analyze_video(self, args):
        """
        Analyse the video file and provide a description of its content.

        :param args: Arguments containing video path.
        :return: Dictionary with video description and path.
        """
        self.node.get_logger().info(f"Executing analyze_video() tool with args: {args}")
        try:
            video_path = args.get("video_path")
            # ... (the smart path fallback mechanism)

            if video_path and os.path.exists(video_path):
                # ... (building the Prompt)

                # Use a fully isolated, one-time context for the video analysis to ensure a
plain text description.
                simple_context = [{
                    "role": "system",
                    "content": "You are a video description assistant. ..."
                }]

                result = self.node.model_client.infer_with_video(video_path, prompt,
message=simple_context)

                # ... (processing the result)
                return {
                    "description": description,
                    "video_path": video_path
```

```
                }
            # ... (the error handling)
```

2. The model interface layer and the frame extraction (`largemodel/utils/large_model_interface.py`)

The functions in this file are responsible for processing the video file and passing it to the underlying model.

```
# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_video(self, video_path, text=None, message=None):
        """Unified video inference interface."""
        # ... (preparing the message)
        try:
            # Depending on the value of self.llm_platform, decide which implementation to
call
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, video_path=video_path)
            # ... (the logic of the other online platforms)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

    def _extract_video_frames(self, video_path, max_frames=5):
        """Extract keyframes from a video for analysis."""
        try:
            import cv2
            # ... (the video reading and the frame interval calculation)
            while extracted_count < max_frames:
                # ... (reading the video frames in a loop)
                if frame_count % frame_interval == 0:
                    # ... (saving the frame as a temporary picture)
                    frame_base64 = self.encode_file_to_base64(temp_path)
                    frame_images.append(frame_base64)
            # ...
            return frame_images
        # ... (the exception handling)
```

Code analysis

The implementation of the video analysis function is one step more complex than the image analysis, because it needs a key preprocessing step in the model interface layer: the frame extraction.

1. **The tool layer (`tools_manager.py`)**:
    - The `analyze_video` function is the business entry point of the video analysis. Its responsibility is very clear: it receives the path of a video file and builds an instruction (Prompt) that asks for a description of the content of the video.
    - It starts the analysis flow by calling the `self.node.model_client.infer_with_video` method. Just like the visual understanding tool, it does not care at all about the underlying model details; it only passes on the "video file" and the "analysis instruction".

2. **The model interface layer (`large_model_interface.py`)**:
    - The `infer_with_video` function is the dispatcher that connects the upper tool and the underlying model. It dispatches the task to the corresponding concrete implementation function according to the platform configuration (`self.llm_platform`).
    - Unlike the handling of a single picture, the handling of a video needs an additional step. The `_extract_video_frames` method shows the general logic for implementing this step: it uses the `cv2` library to read the video and extracts a number of keyframes from it (5 by default).
    - Each extracted frame is treated as an independent picture and is normally encoded as a Base64 string.
    - Finally, the request that contains the data of several frame images and the analysis instruction is sent to the large model. The model performs a comprehensive analysis of these consecutive images and generates a description of the content of the whole video.
    - This "video to multiple pictures" preprocessing is completely encapsulated in the model interface layer and is transparent to the tool layer.

In summary, the general flow of the video analysis is: `ToolsManager` starts the analysis request -> `model_interface` intercepts the request and calls `_extract_video_frames` to break the video file down into several keyframe pictures -> `model_interface` sends these pictures together with the analysis instruction to the corresponding model platform according to the configuration -> the model returns a comprehensive description of the video -> the result is finally handed back to `ToolsManager`. This design ensures the stability and the generality of the upper application.

3. Hands-on operation

3.1 Configure the large model

This section selects the backend through the platform option in `yahboom.yaml` and fills in the corresponding model parameters in `large_model_interface.yaml`. The online example uses Tongyi Qianwen; the offline example uses the local Ollama.

3.1.1 Configure the LLM platform (`yahboom.yaml`)

Open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set `llm_platform` in the existing `model_service.ros__parameters` parameter block. You only have to choose one of the two backends:

```
model_service:
  ros__parameters:
    language: 'en'
    llm_platform: 'ollama'  # The offline large model; change it to 'tongyi' when you use
the online model
    regional_setting: "international"
```

- Offline Ollama: set `llm_platform: 'ollama'`.

- Online Tongyi Qianwen: set `llm_platform: 'tongyi'`.

If these parameters already exist in the file, only modify the corresponding values and keep the other configuration; do not add duplicate YAML keys with the same name.

3.1.2 Configure the model interface (`large_model_interface.yaml`)

Open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Depending on the backend selected in the previous step, only fill in the matching group of parameters.

Online Tongyi Qianwen: first apply for the API Key as described in the online large model basic tutorial and confirm that the account has available quota; then fill in the Key and a model that supports image input:

```
qianwen_api_key: "Replace it with your own API-Key"
qianwen_model: "qwen-vl-max-latest"
```

Replace the placeholder with your own API Key. The video is analysed by extracting the keyframes, so the selected model has to support image input. Never publish a real API Key.

Offline Ollama: make sure the Ollama service is running and that a model supporting image input has been downloaded; then fill in the service address and the name of the local model:

```
ollama_host: "http://localhost:11434"
ollama_model: "llava"  # Replace it with the model name from ollama list
```

The model name has to match the output of `ollama list`, and the model has to support image input.

After saving both configuration files, restart `largemodel` so that the settings take effect. The online mode requires the Jetson to be able to reach the Tongyi service; the offline mode requires the local Ollama service to be available.

> The speed of the video analysis of the offline large model depends on the computing power of the Jetson mainboard and is usually slower than the online large model; the duration of the video, the number of extracted frames and the size of the model also affect the processing time.

3.2 Start it and test the function (text input mode)

1. **Prepare the video file**:

   Put a video file to be tested at the following path:
   `/home/jetson/yahboom_ws/src/largemodel/resources_file/analyze_video`

   Then name the video `test_video.mp4`.

2. **Start the `largemodel` main program (text mode)**:

   Open a terminal and run the following command:

   ```
   ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
   ```

3. **Send the text instruction**:

Open another terminal and run the following command,

```
ros2 run text_chat text_chat
```

Then start typing the text: "analyse this video".

4. **Watch the result**:
   In the first terminal that runs the main program you will see the log output showing that the system received the instruction, called the `analyze_video` tool, extracted the keyframes and finally printed the summary of the video content produced by the AI.

3.3 Voice interaction

The voice mode can use the online or the offline LLM. In `yahboom.yaml`, `llm_platform` is set to `tongyi` for the online and to `ollama` for the offline; the ASR/TTS are chosen independently:

```
model_service:
  ros__parameters:
    use_oline_asr: False  # True: online ASR; False: offline ASR
    useolinetts: False    # True: online TTS; False: offline TTS
```

Complete the corresponding configuration as described in the ASR/TTS tutorials, confirm that the video is in the directory specified above and named `test_video.mp4`, and then start it:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, say "analyse the content of the video". Check the recognised text of the ASR, the log of the video analysis tool call and the result of the model, and confirm that the speaker announces it. The video analysis has to process several frames, so the offline inference takes longer.

4. Frequently asked questions and solutions

Problem 1: it reports "the video file cannot be found" or "the keyframes cannot be extracted from the video".

Solution:

1. **Check the path**: make sure you have placed the video file at the specified path, named it test_video.mp4 and that you have the permission to read the file.
2. **The video format/encoding**: make sure the video file is not corrupted and that it is in the .mp4 format.

Problem 2: analysing a longer video takes a very long time.

Solution:

1. **Use a lighter model**: with limited hardware resources you can switch to a video understanding model with fewer parameters.